MODUL PRAKTIKUM

PEMROGRAMAN BERORIENTASI OBJEK

PENYUSUN:

Timotheus Ferdinand Tjondrojo (NIM: 105222019)

Yan Stephen Christian Immanuel Hutagalung (NIM: 105222010)

LABORATORIUM KOMPUTER

PROGRAM STUDI ILMU KOMPUTER

FAKULTAS SAINS DAN ILMU KOMPUTER

UNIVERSITAS PERTAMINA

2026

Encapsulation

PRE-TEST (Durasi Maksimal: 20 Menit)

Kamu baru saja bergabung sebagai *programmer* di sebuah bank. Saat mengecek kode lama, kamu melihat ada kelas bernama RekeningBank yang membiarkan data nomorRekening dan saldo-nya bebas diakses public! Akibatnya, ada *bug* di mana orang luar bisa tanpa sengaja (atau sengaja) mengubah saldo menjadi angka minus.
Rerombak kelas RekeningBank ini. Terapkan prinsip enkapsulasi agar atribut-atribut penting tersebut aman dan tidak bisa disentuh langsung dari luar kelas. Selanjutnya, buatkan "jalur resmi" (metode khusus) untuk membaca dan mengisi saldo. Pastikan jalur resmi pengisian saldo ini punya penjaga keamanan: saldo hanya mau diperbarui jika angka yang dimasukkan tidak negatif.

1. Access Modifiers

Dalam Java, *access modifiers* adalah mekanisme yang digunakan untuk mengontrol aksesibilitas kelas, variabel, konstruktor, dan metode. Dengan menggunakan *access modifiers*, kita dapat membatasi bagian mana saja dari program yang dapat mengakses suatu member (atribut atau metode) sehingga mendukung prinsip enkapsulasi dan keamanan data pada pemrograman berorientasi objek.
Ada 4 jenis *access modifiers* dalam Java, yakni:

1. Default (tidak diperlukan kata kunci)
2. Private
3. Protected
4. Public

A. Default

Artinya kita tidak mendeklarasikan *access modifier* secara eksplisit untuk class, field, method, konstruktor, dll. Jika tidak ada modifier akses yang dideklarasikan (alias tidak ada public, private, atau protected), maka akses bersifat *default* atau *package-private*. Anggota tersebut hanya dapat diakses oleh kelas-kelas yang berada dalam *package* yang sama.
Contoh:

```
class ContohDefault {
   int angka = 40; // Modifier default (package-private)

   void tampilkanAngka() {
      System.out.println("Angka: " + angka);
   }
}
```

Access modifier default cocok untuk situasi di mana anggota hanya relevan untuk digunakan oleh kelas-kelas dalam satu package saja.

B. Private

Metode, variabel, dan konstruktor yang dideklarasikan sebagai private hanya dapat diakses di dalam kelas yang dideklarasikan itu sendiri. *Access modifiers private* adalah tingkat akses yang paling ketat. Kelas dan interface tidak dapat dideklarasikan sebagai private.
Variabel yang dideklarasikan sebagai private dapat diakses dari luar kelas jika metode getter publik tersedia di dalam kelas tersebut. Penggunaan modifier private adalah cara utama bagi suatu objek untuk melakukan enkapsulasi dirinya sendiri dan menyembunyikan data dari dunia luar.
Contoh:

```
public class ContohPrivat {
   private int angka = 20;

   private void tampilkanAngka() {
      System.out.println("Angka: " + angka);
   }

   public void aksesTampil() {
      tampilkanAngka(); // Boleh dipanggil di dalam kelas yang sama
   }
}
```

C. Protected

Variabel, metode, dan konstruktor yang dideklarasikan sebagai protected dalam sebuah superclass hanya dapat diakses oleh subclass di package lain atau oleh kelas mana pun dalam package yang sama dengan kelas yang memiliki anggota protected. Modifier akses protected tidak dapat diterapkan pada kelas dan antarmuka (interface).
Akses protected memberikan kesempatan kepada subclass untuk menggunakan metode atau variabel pembantu, sekaligus mencegah kelas yang tidak terkait mencoba menggunakannya.
Contoh:

```
public class ContohProtected {
   protected int angka = 30;

   protected void tampilkanAngka() {
      System.out.println("Angka: " + angka);
   }
}

// Kelas turunan di package yang sama atau berbeda
public class Turunan extends ContohProtected {
   public void cetakAngka() {
      // Dapat mengakses anggota protected dari kelas induk
      System.out.println("Angka dari Turunan: " + angka);
```

```
    }
}
```

Penggunaan protected berguna ketika kita ingin agar anggota kelas bisa diakses oleh subclass, namun tidak terbuka untuk semua kelas di luar paket jika tidak berada dalam hubungan pewarisan.

D. Public

Sebuah kelas, metode, konstruktor, antarmuka, dan lainnya yang dideklarasikan sebagai public dapat diakses dari kelas mana pun. Oleh karena itu, field, metode, dan blok yang dideklarasikan di dalam kelas public dapat diakses dari kelas mana pun yang termasuk dalam ekosistem Java.
Namun, jika kelas public yang ingin kita akses berada di package yang berbeda, maka kelas tersebut tetap perlu diimpor terlebih dahulu.
Contoh:

```
public class ContohPublic {
    public int angka = 10;

    public void tampilkanAngka() {
        System.out.println("Angka: " + angka);
    }
}
```

Penggunaan public cocok untuk metode atau variabel yang memang ingin diakses secara luas.

Ringkasan Cakupan Akses

Tabel berikut memberikan gambaran ringkas tentang cakupan akses dari masing-masing access modifiers:

Modifier	**Akses dalam Kelas**	**Akses dalam Package**	**Akses dalam Subclass (beda package)**	**Akses dari Luar Package**
public	✓	✓	✓	✓
protected	✓	✓	✓	✕
default	✓	✓	✕	✕
private	✓	✕	✕	✕

2. Encapsulation dan Information Hiding

Encapsulation (Enkapsulasi) adalah proses mengikat (binding) data dan metode dalam satu unit (kelas), sekaligus membatasi akses langsung ke data tersebut. Hal ini dilakukan agar data dalam suatu objek tetap aman dan tidak bisa diubah sembarangan dari luar kelas.
Information Hiding (Penyembunyian Data) merupakan bagian fundamental dari enkapsulasi yang berfokus pada penyembunyian data internal dari suatu objek dan hanya memperbolehkan akses melalui

metode yang telah ditentukan. Dengan cara ini, data dilindungi dari modifikasi langsung yang tidak diinginkan dari luar kelas, dan pengembang dapat menyisipkan logika validasi atau aturan tertentu ketika data tersebut diubah.
Untuk mengimplementasikan enkapsulasi dan penyembunyian data, biasanya atribut atau data pada kelas dibuat dengan modifier akses private, sehingga tidak bisa diakses secara langsung dari luar kelas.

3. Getter dan Setter

Untuk memberikan akses yang terkendali terhadap data yang telah disembunyikan (private), digunakan metode publik yang disebut **getter** dan **setter**.

- **Getter** adalah metode yang bertugas mengambil atau mengembalikan nilai atribut. Metode ini biasanya diberi nama dengan awalan "get" diikuti nama atribut (misalnya getNama untuk mengambil nilai variabel nama). Getter memungkinkan kelas lain untuk membaca nilai dari atribut private tanpa bisa mengubah nilainya secara langsung.
- **Setter** adalah metode yang digunakan untuk mengubah nilai atribut dengan cara yang terkontrol. Metode ini biasanya diberi nama dengan awalan "set" diikuti nama atribut (misalnya setNama untuk mengubah nilai variabel nama). Di dalam setter, kita bisa menambahkan logika validasi—misalnya memastikan bahwa nilai yang diberikan valid—sebelum menetapkan nilai baru pada atribut tersebut.

Contoh Implementasi Enkapsulasi, Information Hiding, Getter & Setter

```
public class Mahasiswa {
    // Atribut privat: tidak dapat diakses langsung dari luar kelas (Information Hiding)
    private String nama;
    private int umur;

    // Konstruktor untuk menginisialisasi objek Mahasiswa
    public Mahasiswa(String nama, int umur) {
        this.nama = nama;
        this.umur = umur;
    }

    // Getter untuk mengambil nilai atribut 'nama'
    public String getNama() {
        return nama;
    }

    // Setter untuk mengubah nilai atribut 'nama'
    public void setNama(String nama) {
        this.nama = nama;
    }

    // Getter untuk mengambil nilai atribut 'umur'
```

```
    public int getUmur() {
        return umur;
    }

    // Setter untuk mengubah nilai atribut 'umur' dengan validasi
    public void setUmur(int umur) {
        if (umur > 0) { // Validasi: umur harus lebih besar dari 0
            this.umur = umur;
        } else {
            System.out.println("Nilai umur tidak valid.");
        }
    }

    // Metode untuk menampilkan informasi mahasiswa
    public void tampilkanInfo() {
        System.out.println("Nama: " + nama + ", Umur: " + umur);
    }
}
```

Penjelasan Kode di Atas:

- **Atribut Privat:** Variabel nama dan umur dideklarasikan sebagai private sehingga hanya dapat diakses dari dalam kelas itu sendiri. Ini mencegah modifikasi langsung yang bisa merusak integritas data.
- **Konstruktor:** Digunakan untuk menginisialisasi nilai awal atribut ketika objek Mahasiswa dibuat.
- **Getter dan Setter:** * getNama() dan getUmur() memberikan akses baca terhadap atribut.
    - setNama() dan setUmur() memberikan akses ubah dengan tambahan validasi. Misalnya, dalam setUmur(), hanya nilai umur yang positif yang diterima, sehingga data tidak akan diubah dengan nilai yang tidak valid.

Contoh Penggunaannya dalam Main Class:

```
public class Main {
    public static void main(String[] args) {
        // Membuat objek Mahasiswa
        Mahasiswa mhs = new Mahasiswa("Andi", 20);

        // Mengakses dan menampilkan data melalui getter
        System.out.println("Nama awal: " + mhs.getNama());
        System.out.println("Umur awal: " + mhs.getUmur());

        // Mengubah nilai dengan setter
        mhs.setNama("Budi");
        mhs.setUmur(22);

        // Menampilkan informasi setelah perubahan
```

```
        mhs.tampilkanInfo();

        // Mencoba mengatur umur dengan nilai tidak valid
        mhs.setUmur(-5); // Akan menghasilkan pesan error validasi
    }
}
```

LATIHAN PRAKTIKUM (Studi Kasus)

Soal 1: Buat *blueprint* untuk data karyawan perusahaanmu dengan membuat sebuah class bernama Karyawan. Di dalam sistem ini, setiap karyawan wajib memiliki identitas dasar berupa idKaryawan dan nama (keduanya bertipe String). Karena data ini cukup krusial, kamu harus memastikan keamanannya dengan memberikan hak akses private agar tidak bisa diotak-atik secara langsung oleh orang luar. Untuk mempermudah pendaftaran karyawan baru, sediakan sebuah konstruktor sehingga ID dan nama bisa langsung diisi sejak awal pendaftaran. Tentu saja, sistem tetap butuh cara untuk membaca dan memperbarui data tersebut secara terkontrol, jadi jangan lupa buatkan jalur resmi berupa Getter dan Setter untuk kedua atribut tersebut agar tetap aman.

Soal 2: Kamu ditugaskan untuk menambahkan data gajiPokok (bertipe double) ke dalam kelas Karyawan-mu. Sama seperti identitas dasar, informasi keuangan ini sangat sensitif dan harus ditutup rapat-rapat dari akses luar menggunakan hak akses private. Agar tidak terjadi kesalahan fatal, kamu harus menambahkan sedikit logika keamanan ekstra. Saat kamu membuat metode Setter untuk gajiPokok, pastikan sistem menolak perubahan jika nominal yang dimasukkan kurang dari nol, lalu cetak pesan teguran. Buatkan juga metode Getter-nya agar bagian keuangan di sistem nanti tetap bisa melihat rincian gaji tersebut saat dibutuhkan.

Soal 3: Startup kamu kini semakin besar dan terstruktur, sehingga karyawan mulai dikelompokkan ke dalam berbagai divisi. Tambahkan atribut departemen (bertipe String) ke kelas Karyawan milikmu. Berbeda dengan data sebelumnya, kali ini atur hak aksesnya sedikit lebih longgar, yakni menggunakan protected. Tujuannya agar data ini masih relatif aman dari class luar sembarangan, namun tetap praktis jika nantinya diakses oleh class lain yang masih berada dalam satu paket (package) kerja yang sama. Setelah semua data lengkap, pimpinan ingin melihat rangkuman profil karyawannya dengan mudah. Buatlah sebuah fitur di dalam class tersebut bernama tampilkanDataKaryawan() yang bertugas mencetak seluruh informasi mulai dari ID, nama, departemen, hingga gaji pokok ke layar secara rapi.

Soal 4: Buatlah kelas utama bernama Main. Cobalah simulasikan proses penerimaan karyawan baru dengan menginstansiasi satu objek Karyawan. Gunakan Setter untuk mengisi data departemennya. Nah, sekarang coba tes keamanan sistemmu: coba memasukkan gaji pokok dengan angka minus. Terakhir, perbaiki input gajinya dengan angka yang benar (positif), lalu suruh sistem untuk memamerkan profil lengkap karyawan tersebut dengan memanggil metode tampilkanDataKaryawan().

POST-TEST (Durasi: 40 - 50 Menit)

Studi Kasus Terintegrasi: Menyelamatkan Gudang Elektronik

Kamu adalah Lead Programmer di toko elektronik. Masalah terbesar mereka saat ini adalah stok barang yang sering kacau balau karena sistem lama membiarkan siapa saja mengubah stok dan harga sesuka hati. Bosmu menugaskanmu untuk membereskan kekacauan ini menggunakan ilmu yang sudah dipelajari.

Pertama, buatlah kelas Barang untuk mendata idBarang, namaBarang, stok, dan hargaSatuan. Kunci rapat-rapat semua data ini (private) agar tidak ada yang bisa memodifikasinya secara diam-diam. Buat sebuah konstruktor di mana saat barang baru dicatat, cukup masukkan ID dan namanya saja, sementara stok dan harganya otomatis diatur ke angka 0.

Selanjutnya, buatkan Getter dan Setter untuk semuanya, tapi dengan aturan ekstra ketat: Stok sama sekali tidak boleh diset dengan angka negatif. Harga satuan tidak boleh diset kurang dari atau sama dengan nol. Lalu, tambahkan fitur tambahStok dan kurangiStok. Aturannya simpel tapi penting: jumlah yang ditambah atau dikurangi harus lebih dari nol, dan saat stok dikurangi, pastikan jumlah di gudang mencukupi (gamungkin kan stok itu negatif).

Kedua, toko ini juga ingin mengklasifikasikan barangnya (misalnya: "Elektronik", "Pecah Belah", dll). Tambahkan atribut kategori ke dalam kelas Barang tersebut. Atur hak akses atribut ini menjadi protected. Di dalam kelas Barang ini juga, buatlah fitur tampilkanDetailBarang yang akan memunculkan informasi super lengkap ke layar (ID, nama, kategori, stok, dan harga).

Terakhir, buat kelas utama Main. Buat satu objek Barang. Coba buat sebaris kode yang memaksa mengubah stok secara langsung tanpa lewat Setter (misal: barang.stok = 50;). Tuliskan penjelasan singkat dengan komentar. Lanjutkan demomu dengan mencoba merusak sistem: masukkan harga minus dan kurangi stok melebihi batas . Akhiri dengan menampilkan semua detail barang tersebut setelah diisi dengan data yang benar.